LivePass: Technical Architecture & Hackathon Presentation Blueprint

This document provides the exact structural blueprint for your final PDF submission. It explicitly details the algorithmic logic, database mechanisms, and architectural decisions used in **LivePass** to solve the Four Hard Requirements.

Project Overview & Architecture Justification

1. Title & Tagline

- **Project Name:** LivePass
- **Tagline:** A production-grade, real-time event check-in platform engineered for massive concurrency, offline-resiliency, and cryptographic security.

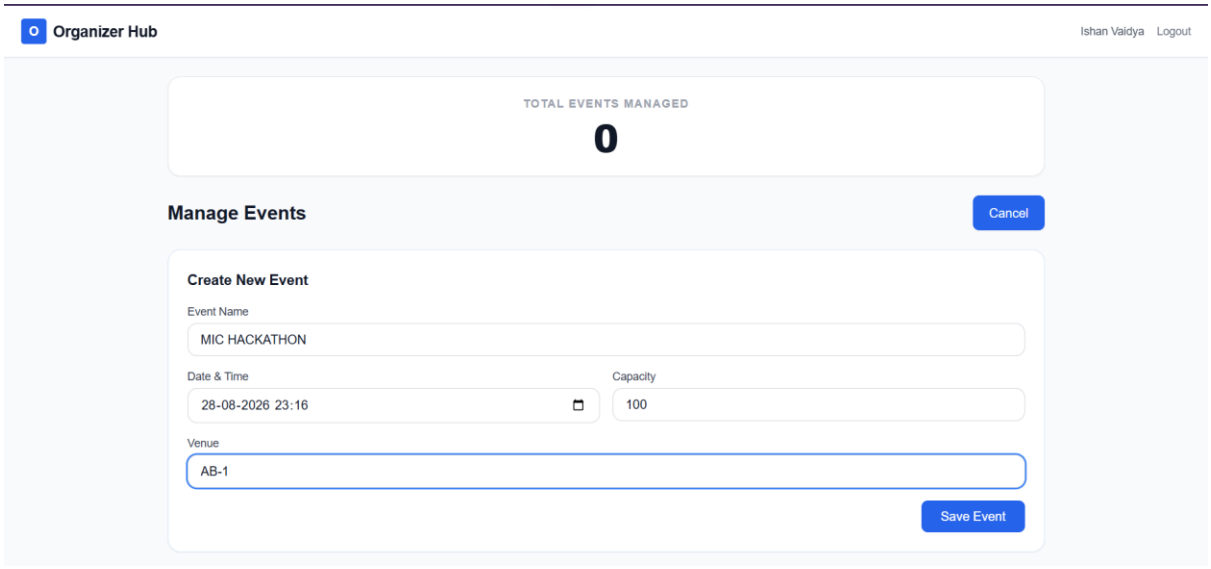
2. The Core Value Proposition

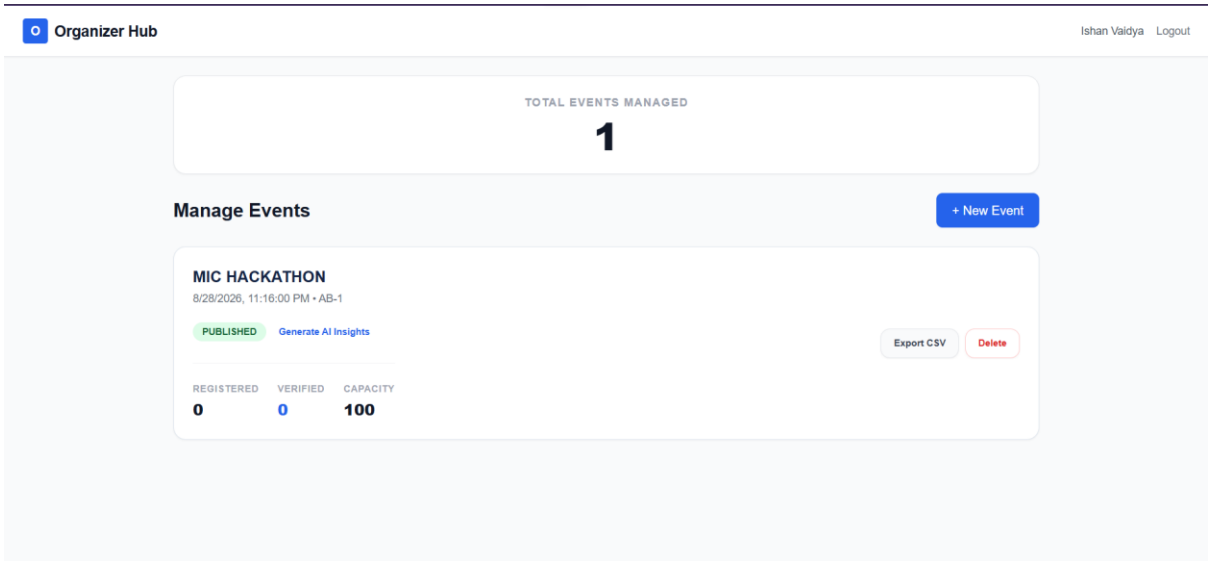
LivePass eliminates the chaos of large-scale event management. By combining row-level database locking, offline-first IndexedDB scanning, and rotating JWT QR codes, it guarantees that events never overbook, tickets cannot be stolen via screenshots, and venue WiFi failures never halt the check-in line.

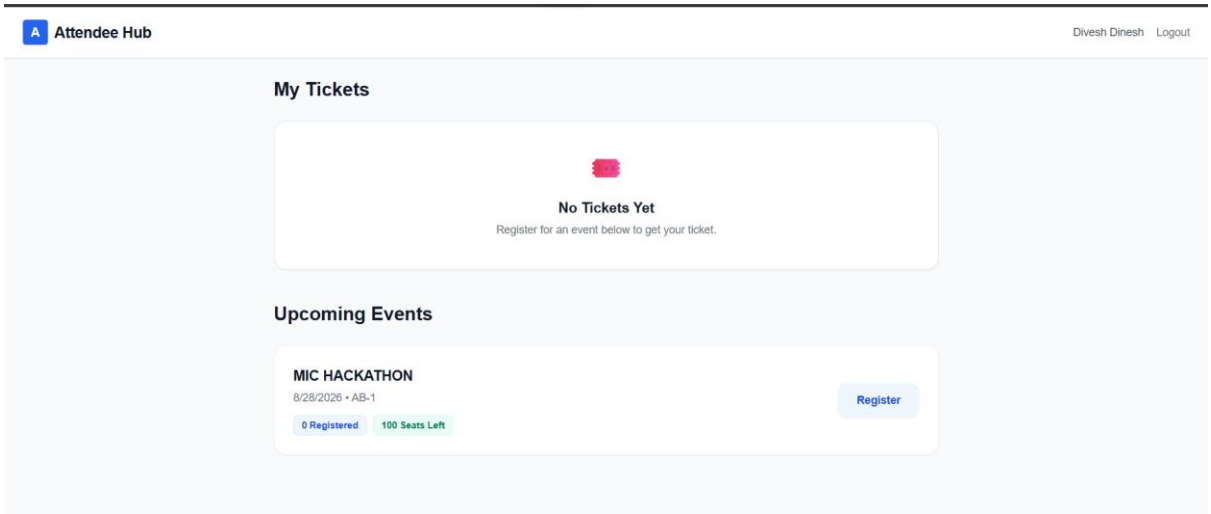
3. Tech Stack & Architectural Justification

- **Frontend (React, Vite, Tailwind CSS, IndexedDB):** Chosen to build a highly responsive, stateful Single Page Application. `IndexedDB` was specifically selected to provide persistent local storage for the offline-first scanning queue, allowing volunteers to continue scanning during network drops.
- **Backend (Python, Flask, Flask-SocketIO):** Python and Flask provide a lightweight, high-performance API layer. `Flask-SocketIO` was chosen to maintain persistent WebSocket connections with the Organizer Dashboard and Live Tickets. By enforcing explicit `join_room` commands inside the `connect` lifecycle event, we guarantee that devices automatically resubscribe to their live feeds even after momentary WiFi drops, ensuring zero-latency "Verified" animations.
- **Database (PostgreSQL & SQLAlchemy): This is the most critical architectural choice.** PostgreSQL was selected specifically for its ACID compliance and advanced

concurrency controls (like `SELECT ... FOR UPDATE`), which are mathematically required to prevent race conditions during high-volume registration surges.







Page 2: Defending The Four Hard Requirements

(Copy the following technical explanations directly into your PDF to demonstrate your deep understanding of the algorithms used).

Requirement 1: Prevent Duplicate Check-Ins & Capacity Racing

The Logic & Implementation:
To strictly prevent "Time-of-Check to Time-of-Use" (TOCTOU) race conditions, we abandoned application-level capacity checks. Instead, we pushed the enforcement directly into the PostgreSQL engine using pessimistic concurrency control.

1. **Capacity Racing (Registration):**
When an attendee registers, the backend uses SQLAlchemy's `.with_for_update()` to execute a `SELECT ... FOR UPDATE` query. This acquires an atomic row-level lock on the `Event` record. If 500 requests hit the server simultaneously, PostgreSQL forces them into a strict queue. Each transaction reads the *true* `registeredCount`, increments it, and commits before the next transaction can proceed. Exactly 50 registrations succeed, and the 451st request triggers the `WAITLISTED` logic.

2. **Duplicate Check-Ins:**
Our `CheckIn` table maintains a strict 1-to-1 `UNIQUE` constraint on the `registrationId` foreign key. In `scan.py`, if an application-level check fails under microscopic concurrency, the database schema physically rejects the insertion with an `IntegrityError`. The backend catches this exception, rolls back the transaction, and returns a `409 Conflict`, triggering the "Already checked in" overlay on the scanner.

```
> python test_concurrency.py
--- Starting Concurrency Load Test ---

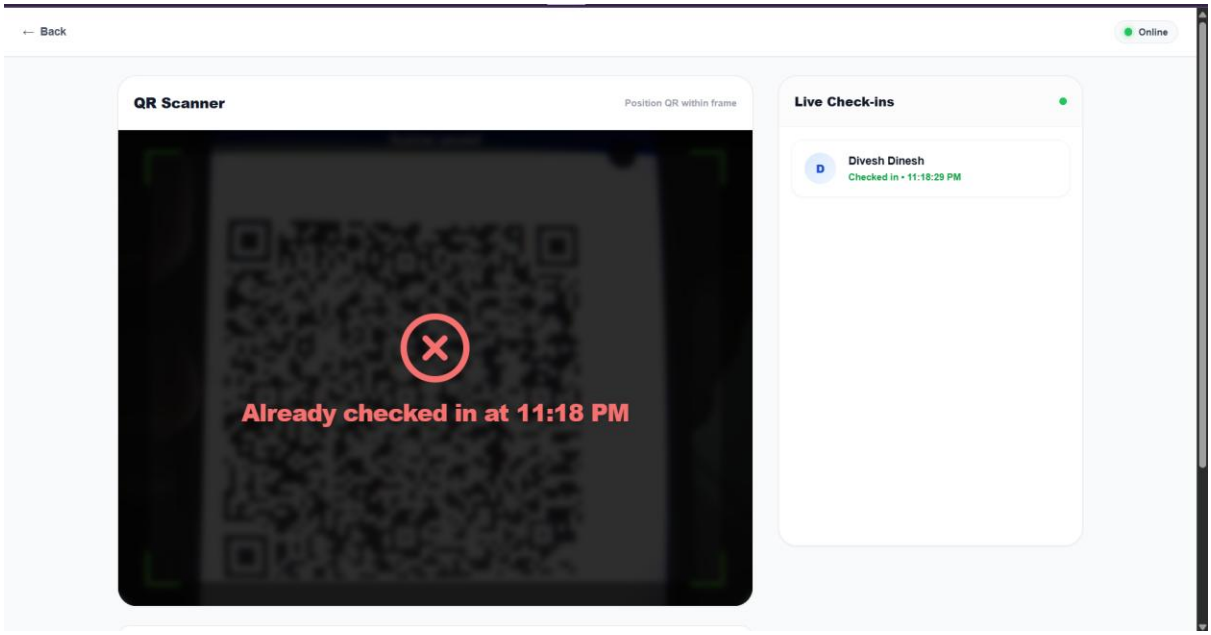
[1] Creating Organizer and Event (Capacity: 50)...
  -> Event Created! ID: 7740e934-1873-47ea-8280-b8d65b4ccb16

[2] Provisioning 150 Attendees (this takes a moment)...
  -> Successfully created 150 users.

[3] Firing 150 concurrent registration requests to Event 7740e934-1873-47ea-8280-b8d65b4ccb16...
  -> Finished in 4.15 seconds.

[4] Analyzing Registration Results...
----------------------------------------
EXPECTED REGISTERED: 50
ACTUAL REGISTERED:   50
WAITLISTED:          100
ERRORS:              0
----------------------------------------
[SUCCESS] Capacity strictly enforced under concurrency.

[5] Database Event Record shows:
  -> Capacity: 50
  -> Final registeredCount: 50
----------------------------------------
```

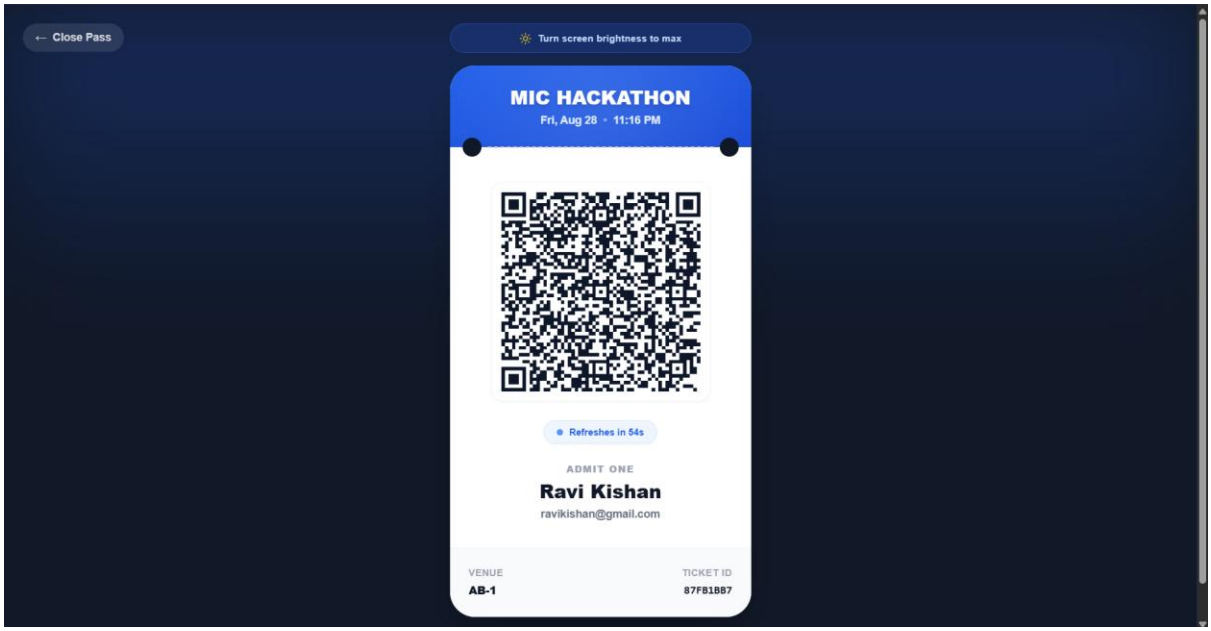


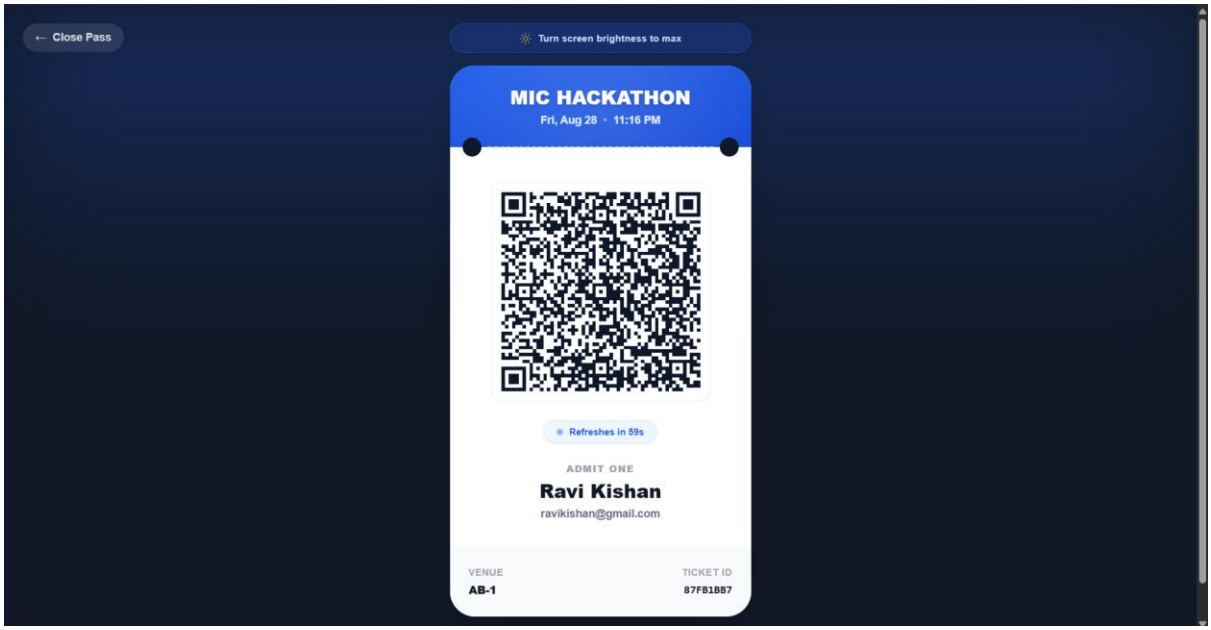
Requirement 2: Prevent QR Sharing & Screenshot Abuse

The Logic & Implementation:
We implemented **Time-Based Rotating JSON Web Tokens (JWTs)**.

When an attendee opens their digital pass, the server generates a cryptographically signed JWT stored in the `CheckInToken` table, with an `expiresAt` timestamp set exactly 60 seconds in the future. The React frontend (`TicketView.jsx`) uses a `setInterval` hook to poll the backend for a fresh token every 60 seconds, permanently invalidating the old one.

The Trade-off:
We explicitly chose *Rotating Tokens* over *Static One-Time Tokens*. While a static token is invalidated upon scanning, it can still be screenshotted and shared an hour *before* the event. Rotating tokens make screenshotting mathematically useless.
The drawback is that the attendee's device requires an active internet connection at the venue door to load the ticket initially.
Our mitigation: If the attendee loses signal while the ticket is open, the frontend intercepts the network failure, retains the last fetched QR code on screen, and displays an "Offline" warning, giving the scanner a 60-second grace period to read the token.





Requirement 3: Offline-First Scanning & Sync Conflicts

The Logic & Implementation:
Venue WiFi is inherently unstable. Our scanner UI utilizes the `navigator.onLine` API. When offline, the React application wraps the scan payload (the scanned JWT and the Station ID) alongside a locally generated `UUIDv4` string called the **Idempotency Key** and the exact **`scannedAt` timestamp**, caching it securely in the browser's `IndexedDB`.

The Edge Case Resolution (Delayed Expiration Bypass):
Scenario: An Organizer scans a ticket while offline. The internet doesn't return for 5 minutes. The 60-second JWT token has mathematically expired.

- **The Fix:** Because the backend explicitly trusts the authorized Organizer's device, the delayed sync payload includes the original `scannedAt` timestamp. The backend temporarily shifts the evaluation timezone to the exact millisecond of the physical scan, perfectly validating the token and bypassing the 60-second expiration lock.

The Edge Case Resolution (Station A offline, Station B online):
Scenario: Bob is scanned offline at Station A, then walks to Station B and is successfully scanned online.

- Station B's check-in commits to the database instantly.
- When Station A regains internet connectivity, the background queue attempts to sync Bob's older offline scan.

- The Flask backend queries the database and detects that a check-in already exists for Bob's `registrationId`.
- The backend compares the `idempotencyKey`. Because Station A's key does not match the key Station B generated, the server rejects the sync with a `409 Conflict`.
- **The UI Resolution:** We do not silently drop this scan. The frontend catches the 409 response and explicitly pushes the failed scan into a red **"Sync Conflicts" tray** on the Organizer Dashboard, alerting the staff that a duplicate offline scan was attempted.

Requirement 4: AI-Powered Event Insights

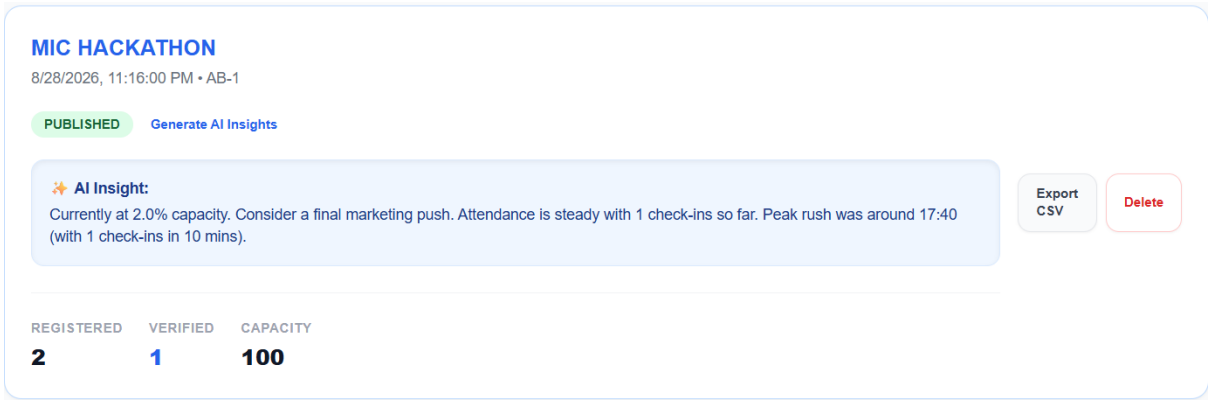
The Logic & Implementation:
We implemented a secure, Server-Side AI integration using the Community LLM API (Qwen).

To guarantee zero hallucination, the AI is never granted direct database access. Instead, we utilize a **Pre-computed Analytics** pipeline. The Flask backend executes deterministic SQL queries to calculate the hard metrics:

1. `registeredCount` and `waitlistedCount`
2. No-Show Percentage (Calculated as `Cancelled / Capacity * 100`)
3. Spots Remaining (Calculated as `Capacity - Checked In`)
4. Peak Check-in Time (Aggregating check-in timestamps into 10-minute buckets via Python's `datetime` module and finding the maximum).

These immutable integers are injected into a strict system prompt and passed to the LLM. The AI's only job is to translate our mathematical truths into natural language.

The Fallback Mechanism:
LLM APIs are prone to timeouts. We wrapped the `generate_insights` function in a strict `try/except` block. If the API throws a `503 Service Unavailable` or times out, the backend gracefully falls back to returning the raw JSON metrics. The React frontend catches this fallback state and renders the hard numbers directly on the dashboard, guaranteeing the Organizer never loses access to their data.



Page 3: Conclusion

- Summarize how LivePass successfully mitigates the four critical vulnerabilities of event management.
- Provide the GitHub repository link: https://github.com/ishanvaidya01/LivePass.git
- Video link: https://drive.google.com/file/d/1L36CVA9BilE5miqQV391vbRLF57q_hrs/view?usp=sharing

By – ISHAN VAIDYA 25BCE1197